Hashing: funções aleatórias, colisões e universalidade

J. D.

UFABC

Novembro 2, 2026

Roteiro

1. **Funções de hash** e por que precisamos de aleatoriedade
2. **Tabelas de espalhamento** e custo de busca com função aleatória
3. **A maior lista**: cota $O(\log n / \log \log n)$
4. **Paradoxo dos aniversários** e o limiar $\sqrt{n}$
5. **Variância** e independência par a par
6. **Síntese**: o que basta para famílias universais
7. **Famílias universais**: definição, exemplo linear e $h_{(a,b)}$
8. **Aplicação**: *Count-Min sketch*
7. **Famílias 2-universais**: definição e exemplos
8. **Aplicação**: "maxcut" e desaleatorização

Funções de hash

O que é uma função de hash?

Definição 1

Uma função de hash é uma função $h\colon U \to N$, onde o universo U é *muito grande* e $N = \{0, 1, \ldots, n-1\}$.

- $h(x)$ é um "resumo" de x: ocupa pouco espaço, é comparável e serve de índice em tempo constante.
- Como $|U| \gg |N|$, há colisões: $x \neq y$ com $h(x) = h(y)$. Pelo princípio da casa dos pombos, não dá para evitá-las.

O que é uma função de hash?

Definição 1

Uma função de hash é uma função $h\colon U \to N$, onde o universo U é *muito grande* e $N = \{0, 1, \ldots, n-1\}$.

- $h(x)$ é um "resumo" de x: ocupa pouco espaço, é comparável e serve de índice em tempo constante.
- Como $|U| \gg |N|$, há colisões: $x \neq y$ com $h(x) = h(y)$. Pelo princípio da casa dos pombos, não dá para evitá-las.

Usos: dicionários (tabelas de espalhamento), impressões digitais e verificação de igualdade, desaleatorização, criptografia (exige mais: colisões difíceis de achar).

O que torna uma função de hash útil?

São propriedades de interesse e que dependem do uso:

(i) **fácil de computar**: $h(x)$ em tempo (quase) constante;

(ii) **descrição curta**: poucos bits para armazenar h;

(iii) **espalhar bem** os elementos de um conjunto $S \subseteq U$: valores $h(x)$, $x \in S$, aproximadamente uniformes em N, poucas colisões.

O que torna uma função de hash útil?

São propriedades de interesse e que dependem do uso:

(i) **fácil de computar**: $h(x)$ em tempo (quase) constante;

(ii) **descrição curta**: poucos bits para armazenar h;

(iii) **espalhar bem** os elementos de um conjunto $S \subseteq U$: valores $h(x)$, $x \in S$, aproximadamente uniformes em N, poucas colisões.

O problema com (iii)

Para *qualquer* h fixa, se $|U| \geq mn$, existe S com $|S| = m$ em que todos colidem: alguma pré-imagem tem tamanho $\geq |U|/n \geq m$. Nenhuma h determinística espalha bem *todos* os conjuntos.

O que torna uma função de hash útil?

São propriedades de interesse e que dependem do uso:

(i) **fácil de computar**: $h(x)$ em tempo (quase) constante;

(ii) **descrição curta**: poucos bits para armazenar h;

(iii) **espalhar bem** os elementos de um conjunto $S \subseteq U$: valores $h(x)$, $x \in S$, aproximadamente uniformes em N, poucas colisões.

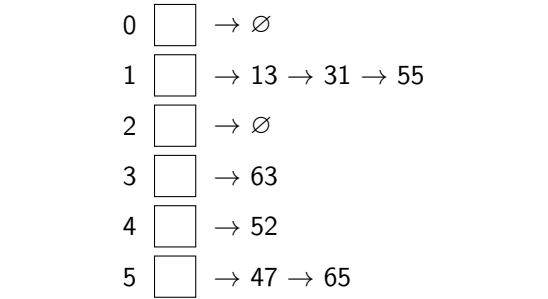
O problema com (iii)

Para *qualquer* h fixa, se $|U| \geq mn$, existe S com $|S| = m$ em que todos colidem: alguma pré-imagem tem tamanho $\geq |U|/n \geq m$. Nenhuma h determinística espalha bem *todos* os conjuntos.

Saída: sortear h de uma família $\mathcal{H}$ e analisar o comportamento *esperado*, ou com alta probabilidade, para qualquer S *fixo*.

Tabela de espalhamento (com listas ligadas)

- Vetor H com n posições; cada $H[i]$ é uma lista ligada.
- Acesso via $h\colon U \to \{0, 1, \ldots, n-1\}$: o elemento x vive na lista $H[h(x)]$.
- $S \subseteq U$ com $m = |S|$; o caso interessante é $m = \Theta(n)$.
- Um vetor de bits de tamanho $|U|$ daria busca $O(1)$, mas é inviável.



$S = \{13, 31, 47, 52, 55, 63, 65\}$, $h(x) = x \bmod 6$

Operações e custos

Inserção colocar x no início de $H[h(x)]$: custo constante.

Remoção (com apontador para x, em lista duplamente ligada) retirar da lista: custo constante.

Busca percorrer a lista $H[h(x)]$. Pode ser *com sucesso* ($x \in S$) ou *sem sucesso* ($x \notin S$).

Operações e custos

Inserção colocar x no início de $H[h(x)]$: custo constante.

Remoção (com apontador para x, em lista duplamente ligada) retirar da lista: custo constante.

Busca percorrer a lista $H[h(x)]$. Pode ser *com sucesso* ($x \in S$) ou *sem sucesso* ($x \notin S$).

Quem manda no desempenho?

O pior caso da busca é o tamanho da maior lista. Tudo se reduz a entender os tamanhos das listas, que dependem de como h distribui S.

Operações e custos

Inserção colocar x no início de $H[h(x)]$: custo constante.

Remoção (com apontador para x, em lista duplamente ligada) retirar da lista: custo constante.

Busca percorrer a lista $H[h(x)]$. Pode ser *com sucesso* ($x \in S$) ou *sem sucesso* ($x \notin S$).

Quem manda no desempenho?

O pior caso da busca é o tamanho da maior lista. Tudo se reduz a entender os tamanhos das listas, que dependem de como h distribui S.

Primeiro modelo: h uniforme em N^U. Equivale a sortear, *independentemente* para cada $x \in U$, um valor $h(x)$ uniforme em N.

Função aleatória: probabilidade de colisão e carga

Para todo $x \in U$ e $i \in N$: $\mathbb{P}[h(x) = i] = \frac{1}{n}$.

Função aleatória: probabilidade de colisão e carga

Para todo $x \in U$ e $i \in N$: $\quad \mathbb{P}[h(x) = i] = \dfrac{1}{n}$.

Colisão de um par

Para $x \neq y$, por independência,

$$\mathbb{P}[h(x) = h(y)] = \sum_{i \in N} \mathbb{P}[h(x) = i]\, \mathbb{P}[h(y) = i] = n \cdot \frac{1}{n} \cdot \frac{1}{n} = \frac{1}{n}.$$

Função aleatória: probabilidade de colisão e carga

Para todo $x \in U$ e $i \in N$: $\quad \mathbb{P}[h(x) = i] = \dfrac{1}{n}$.

Colisão de um par

Para $x \neq y$, por independência,

$$\mathbb{P}[h(x) = h(y)] = \sum_{i \in N} \mathbb{P}[h(x) = i]\, \mathbb{P}[h(y) = i] = n \cdot \frac{1}{n} \cdot \frac{1}{n} = \frac{1}{n}.$$

Definição 2

A carga da tabela é $c = m/n$, o número esperado de elementos de S por posição.

Tamanho esperado da lista $H[h(x)]$

O número de itens na lista de x é $\sum_{y \in S} \mathbb{1}_{[h(x)=h(y)]}$, e

$$\mathbb{E}\, \mathbb{1}_{[h(x)=h(y)]} = \begin{cases} 1/n, & y \neq x, \\ 1, & y = x. \end{cases}$$

Tamanho esperado da lista $H[h(x)]$

O número de itens na lista de x é $\sum_{y \in S} \mathbb{1}_{[h(x)=h(y)]}$, e

$$\mathbb{E}\, \mathbb{1}_{[h(x)=h(y)]} = \begin{cases} 1/n, & y \neq x, \\ 1, & y = x. \end{cases}$$

Pela linearidade da esperança,

$$\mathbb{E}\left|H[h(x)]\right| = \begin{cases} \dfrac{m}{n} = c, & x \notin S, \\ \dfrac{m-1}{n} + 1 = c + \left(1 - \dfrac{1}{n}\right), & x \in S. \end{cases}$$

Tamanho esperado da lista $H[h(x)]$

O número de itens na lista de x é $\sum_{y \in S} \mathbb{1}_{[h(x)=h(y)]}$, e

$$\mathbb{E}\, \mathbb{1}_{[h(x)=h(y)]} = \begin{cases} 1/n, & y \neq x, \\ 1, & y = x. \end{cases}$$

Pela linearidade da esperança,

$$\mathbb{E}\left|H[h(x)]\right| = \begin{cases} \dfrac{m}{n} = c, & x \notin S, \\ \dfrac{m-1}{n} + 1 = c + \left(1 - \dfrac{1}{n}\right), & x \in S. \end{cases}$$

Conclusão

Com h aleatória em N^U, o tempo esperado de busca é proporcional à carga $O(1)$ quando $n = \Theta(m)$.

Custo esperado das buscas

Proposição 1

Seja H com n posições e h aleatória; elementos inseridos no início das listas. Então

1. *busca sem sucesso: tempo esperado $c + 1$;*
2. *busca com sucesso: tempo esperado $\frac{c}{2} + 1 - \frac{1}{2n}$.*

Custo esperado das buscas

Proposição 1

Seja H com n posições e h aleatória; elementos inseridos no início das listas. Então

1. *busca sem sucesso: tempo esperado $c + 1$;*
2. *busca com sucesso: tempo esperado $\dfrac{c}{2} + 1 - \dfrac{1}{2n}$.*

Ideia da prova do item 2. Se x foi o i-ésimo inserido, só os $m - i$ elementos inseridos *depois* podem ficar na frente de x; em média $(m - i)/n$ deles colidem com x. Logo

$$\frac{1}{m}\sum_{i=1}^{m}\left(1 + \frac{m-i}{n}\right) = 1 + \frac{m-1}{2n} = 1 + \frac{c}{2} - \frac{1}{2n}.$$

Qual é o problema?

O que ganhamos

- busca esperada $O(1)$;
- maior lista $O(\log m / \log \log m)$ com alta probabilidade (a seguir).

O que perdemos

- uma função aleatória em N^U precisa de $\approx |U| \log n$ bits para ser descrita;
- guardar h custa mais que um vetor de bits (vetor característico).

Qual é o problema?

O que ganhamos

- busca esperada $O(1)$;
- maior lista $O(\log m / \log\log m)$ com alta probabilidade (a seguir).

O que perdemos

- uma função aleatória em N^U precisa de $\approx |U| \log n$ bits para ser descrita;
- guardar h custa mais que um vetor de bits (vetor característico).

Pergunta

Das propriedades que usamos até aqui, quais dependem de h ser *totalmente aleatória*?

Vamos acompanhar isso ao longo das próximas análises.

A maior lista é sublogarítmica

Lema 3
Seja $X_i = |\{\, y \in S : h(y) = i \,\}|$, $c = m/n \leq 1$ *e* $n \geq 3$. *Então*

$$\mathbb{P}\Big[\max_i X_i \;\geq\; \frac{8 \ln n}{\ln \ln n}\Big] \;\leq\; \frac{1}{n}.$$

Como $n = \Theta(m)$, a maior lista tem tamanho $O(\log m / \log \log m)$ com probabilidade $\geq 1 - 1/n$.

Prova (1): cota para uma posição

Se $X_i \geq k$, existe $T \subseteq S$, $|T| = k$, com $h(y) = i$ para todo $y \in T$. Pela independência, cada T tem probabilidade n^{-k}. Pela cota da união,

$$\mathbb{P}[X_i \geq k] \leq \binom{m}{k} \frac{1}{n^k} \leq \left(\frac{em}{kn}\right)^k = \left(\frac{ec}{k}\right)^k \leq \left(\frac{e}{k}\right)^k$$

Cota da união, novamente, agora sobre as n listas (*não* precisamos que sejam independentes):

$$\mathbb{P}\big[\max_i X_i \geq k\big] \leq n\left(\frac{e}{k}\right)^k = n\, e^{-\phi(k)}, \qquad \phi(k) := k(\ln k - 1).$$

Estimação de k

Queremos k com $\phi(k) \geq 2 \ln n$, para que a cota seja $\leq 1/n$. Como $k \ln k$ domina k, basta resolver

$$k \ln k \approx 2 \ln n$$

isolando k em termos de n para n grande.

Heurística k deve crescer com n, mas de forma muito mais lenta que n (pois k^k cresce super-exponencialmente), testamos um chute educado para k:

$$k = \frac{a \ln n}{f(n)}$$

onde $1 \ll f(n) \ll \ln n$ é alguma função de crescimento lento e a é uma constante.

Calculando $\ln k$

$$k = \frac{a \ln n}{f(n)} \quad \Longrightarrow \quad \ln k = \ln a + \ln(\ln n) - \ln(f(n))$$

Para n grande, o termo dominante em $\ln k$ é $\ln(\ln n)$, logo

$$\ln k \approx \ln \ln n$$

e

$$k \ln k \approx \left(\frac{a \ln n}{f(n)} \right) (\ln \ln n)$$

de volta à estimação da probabilidade

$$\frac{a \ln n \cdot \ln \ln n}{f(n)} \approx 2 \ln n \quad \Longrightarrow \quad \frac{a \ln \ln n}{f(n)} \approx 2 \quad \Longrightarrow \quad f(n) = \Theta(\ln \ln(n))$$

Prova (2): escolha de k

Tome $k_0 = \dfrac{8 \ln n}{\ln \ln n}$, $k = \lceil k_0 \rceil$, $y = \ln \ln n$

Prova (2): escolha de k

Tome $k_0 = \dfrac{8 \ln n}{\ln \ln n}$, $k = \lceil k_0 \rceil$, $y = \ln \ln n$

- $\ln k_0 - 1 = \ln(8) - 1 + y - \ln y \;\geq\; \ln(8) - 1 + \left(1 - \frac{1}{e}\right) y \;\geq\; \frac{1}{2} y$, pois $\ln(8) - 1 > 0$ e $1 - 1/e \approx 0{,}63 \geq \frac{1}{2}$. Isso vale para todo $y > 0$.

Logo

$$\phi(k) \geq \phi(k_0) = k_0(\ln k_0 - 1) \geq \frac{8 \ln n}{y} \cdot \frac{y}{2} = 4 \ln n,$$

e portanto

$$\mathbb{P}\big[\max_i X_i \geq k\big] \leq n\, e^{-\phi(k)} \leq n\, e^{-4 \ln n} = \frac{1}{n^2}. \qquad \blacksquare$$

Observações sobre a cota

- **Carga constante.** Se $c \leq C$, a cota vira $(Ce/k)^k$; basta trocar e por Ce e ajustar a constante 8.

Observações sobre a cota

- **Carga constante.** Se $c \leq C$, a cota vira $(Ce/k)^k$; basta trocar e por Ce e ajustar a constante 8.
- **A cota é justa.** Um argumento de segundo momento sobre o número de listas com $\geq k$ elementos mostra que, com alta probabilidade, a maior lista tem tamanho $\Omega(\log n / \log\log n)$. Portanto

$$\max_i X_i = \Theta\Big(\frac{\log n}{\log\log n}\Big).$$

Observações sobre a cota

- **Carga constante.** Se $c \leq C$, a cota vira $(Ce/k)^k$; basta trocar e por Ce e ajustar a constante 8.
- **A cota é justa.** Um argumento de segundo momento sobre o número de listas com $\geq k$ elementos mostra que, com alta probabilidade, a maior lista tem tamanho $\Omega(\log n / \log\log n)$. Portanto

$$\max_i X_i = \Theta\Big(\frac{\log n}{\log\log n}\Big).$$

- **O que foi usado?** A independência de k valores $h(y)$ ao mesmo tempo (probabilidade n^{-k} para cada T).

Este resultado precisa de "muita independência".

Número esperado de colisões

Seja C o número de pares $\{x, y\} \subseteq S$ que colidem: $C = \sum_{\{x,y\}} \mathbb{1}_{[h(x)=h(y)]}$. Por linearidade,

$$\mathbb{E}\, C = \binom{m}{2} \frac{1}{n} = \frac{m(m-1)}{2n}.$$

Número esperado de colisões

Seja C o número de pares $\{x, y\} \subseteq S$ que colidem: $C = \sum_{\{x,y\}} \mathbb{1}_{[h(x)=h(y)]}$. Por linearidade,

$$\mathbb{E}\, C = \binom{m}{2} \frac{1}{n} = \frac{m(m-1)}{2n}.$$

Pela desigualdade de Markov,

$$\mathbb{P}[C = 0] \;\geq\; 1 - \mathbb{E}\, C = 1 - \frac{m(m-1)}{2n}.$$

Em particular, para $m = \sqrt{n}$: $\mathbb{E}\, C < \frac{1}{2}$ e $\mathbb{P}[C = 0] \geq \frac{1}{2}$.

Número esperado de colisões

Seja C o número de pares $\{x, y\} \subseteq S$ que colidem: $C = \sum_{\{x,y\}} \mathbb{1}_{[h(x)=h(y)]}$. Por linearidade,

$$\mathbb{E}\, C = \binom{m}{2} \frac{1}{n} = \frac{m(m-1)}{2n}.$$

Pela desigualdade de Markov,

$$\mathbb{P}[C = 0] \;\geq\; 1 - \mathbb{E}\, C = 1 - \frac{m(m-1)}{2n}.$$

Em particular, para $m = \sqrt{n}$: $\mathbb{E}\, C < \frac{1}{2}$ e $\mathbb{P}[C = 0] \geq \frac{1}{2}$.

Observação

Só usamos informação sobre pares.

Probabilidade de nenhuma colisão: fórmula exata

Para $m \leq n$ há $n(n-1)\cdots(n-m+1)$ sequências sem repetição em N^m, logo

$$\mathbb{P}[C = 0] = \frac{n(n-1)\cdots(n-m+1)}{n^m} = \prod_{j=1}^{m-1}\Big(1 - \frac{j}{n}\Big).$$

Probabilidade de nenhuma colisão: fórmula exata

Para $m \leq n$ há $n(n-1)\cdots(n-m+1)$ sequências sem repetição em N^m, logo

$$\mathbb{P}[C = 0] = \frac{n(n-1)\cdots(n-m+1)}{n^m} = \prod_{j=1}^{m-1}\Big(1 - \frac{j}{n}\Big).$$

Cota superior

$1 - x < e^{-x}$ para $x > 0$, logo

$$\mathbb{P}[C = 0] < \exp\Big(-\frac{m(m-1)}{2n}\Big).$$

Probabilidade de nenhuma colisão: fórmula exata

Para $m \leq n$ há $n(n-1)\cdots(n-m+1)$ sequências sem repetição em N^m, logo

$$\mathbb{P}[C = 0] = \frac{n(n-1)\cdots(n-m+1)}{n^m} = \prod_{j=1}^{m-1}\Big(1 - \frac{j}{n}\Big).$$

Cota superior

$1 - x < e^{-x}$ para $x > 0$, logo

$$\mathbb{P}[C = 0] < \exp\Big(-\frac{m(m-1)}{2n}\Big).$$

Cota inferior

$1 - x > e^{-2x}$ para $x \in (0, \frac{3}{4}]$. Se $m \leq \frac{3}{4}n$,

$$\mathbb{P}[C = 0] > \exp\Big(-\frac{m(m-1)}{n}\Big).$$

Probabilidade de nenhuma colisão: fórmula exata

Para $m \le n$ há $n(n-1)\cdots(n-m+1)$ sequências sem repetição em N^m, logo

$$\mathbb{P}[C=0] = \frac{n(n-1)\cdots(n-m+1)}{n^m} = \prod_{j=1}^{m-1}\Big(1-\frac{j}{n}\Big).$$

Cota superior

$1-x < e^{-x}$ para $x>0$, logo

$$\mathbb{P}[C=0] < \exp\Big(-\frac{m(m-1)}{2n}\Big).$$

Cota inferior

$1-x > e^{-2x}$ para $x \in (0, \frac{3}{4}]$. Se $m \le \frac{3}{4}n$,

$$\mathbb{P}[C=0] > \exp\Big(-\frac{m(m-1)}{n}\Big).$$

Em ambas, tudo depende de $m(m-1)/n \approx m^2/n$.

Exemplo: o paradoxo dos aniversários

Com $n = 365$ dias e m pessoas (nascimentos uniformes e independentes):

$$\mathbb{P}[C > 0] = 1 - \mathbb{P}[C = 0] > 1 - \exp\Big(-\frac{m(m-1)}{2n}\Big).$$

Exemplo: o paradoxo dos aniversários

Com $n = 365$ dias e m pessoas (nascimentos uniformes e independentes):

$$\mathbb{P}[C > 0] = 1 - \mathbb{P}[C = 0] > 1 - \exp\Big(-\frac{m(m-1)}{2n}\Big).$$

$m = 23$

$$\frac{m(m-1)}{2n} = \frac{506}{730} \approx 0{,}693151 > \ln 2 \approx 0{,}693147 \quad \Longrightarrow \quad \mathbb{P}[C > 0] > \tfrac{1}{2}.$$

Basta uma turma de 23 pessoas para haver aniversário repetido com probabilidade maior que $1/2$.

O limiar $\sqrt{n}$ para colisões

Juntando as duas cotas, para $m \leq \frac{3}{4}n$:

$$\exp\Big(-\frac{m(m-1)}{n}\Big) < \mathbb{P}[C = 0] < \exp\Big(-\frac{m(m-1)}{2n}\Big).$$

O limiar $\sqrt{n}$ para colisões

Juntando as duas cotas, para $m \leq \frac{3}{4}n$:

$$\exp\Big(-\frac{m(m-1)}{n}\Big) < \mathbb{P}[C = 0] < \exp\Big(-\frac{m(m-1)}{2n}\Big).$$

Regime	**Conclusão**	**Cota usada**
$m \leq \sqrt{n}$	$\mathbb{P}[C = 0] > e^{-1} > 0{,}36$	inferior
$m = \Theta(\sqrt{n})$	$\mathbb{P}[C = 0]$ entre duas constantes em $(0, 1)$	ambas
$m = \omega(\sqrt{n})$	$\mathbb{P}[C = 0] \to 0$	superior

O limiar $\sqrt{n}$ para colisões

Juntando as duas cotas, para $m \leq \frac{3}{4}n$:

$$\exp\Big(-\frac{m(m-1)}{n}\Big) < \mathbb{P}[C = 0] < \exp\Big(-\frac{m(m-1)}{2n}\Big).$$

Regime	**Conclusão**	**Cota usada**
$m \leq \sqrt{n}$	$\mathbb{P}[C = 0] > e^{-1} > 0{,}36$	inferior
$m = \Theta(\sqrt{n})$	$\mathbb{P}[C = 0]$ entre duas constantes em $(0, 1)$	ambas
$m = \omega(\sqrt{n})$	$\mathbb{P}[C = 0] \to 0$	superior

Limiar

$\sqrt{n}$ é o limiar: até ele, inserimos sem colisão com probabilidade constante; acima dele, há colisão quase certamente.

O limiar $\sqrt{n}$ para colisões

Juntando as duas cotas, para $m \leq \frac{3}{4}n$:

$$\exp\Big(-\frac{m(m-1)}{n}\Big) < \mathbb{P}[C=0] < \exp\Big(-\frac{m(m-1)}{2n}\Big).$$

Regime	**Conclusão**	**Cota usada**
$m \leq \sqrt{n}$	$\mathbb{P}[C=0] > e^{-1} > 0{,}36$	inferior
$m = \Theta(\sqrt{n})$	$\mathbb{P}[C=0]$ entre duas constantes em $(0,1)$	ambas
$m = \omega(\sqrt{n})$	$\mathbb{P}[C=0] \to 0$	superior

Limiar

$\sqrt{n}$ é o limiar: até ele, inserimos sem colisão com probabilidade constante; acima dele, há colisão quase certamente.

Markov: para $m \leq \sqrt{n}$ dá $\mathbb{P}[C=0] \geq 1 - \mathbb{E}C > \frac{1}{2}$ (melhor que e^{-1}), mas é trivial quando $\mathbb{E}C \geq 1$, isto é, $m(m-1) \geq 2n$ (em particular, se $m \geq \sqrt{2n}+1$).

Variância do tamanho de uma lista

Seja $X_i = \sum_{x \in S} \mathbb{1}_{[h(x)=i]}$, com $\mathbb{E}X_i = c$. Para $x \neq y$ os indicadores são independentes. Então

$$\text{Var}(\, X_i \,) = \sum_{x \in S} \frac{1}{n}\left(1 - \frac{1}{n}\right) = c\left(1 - \frac{1}{n}\right).$$

Variância do tamanho de uma lista

Seja $X_i = \sum_{x \in S} \mathbb{1}_{[h(x)=i]}$, com $\mathbb{E}X_i = c$. Para $x \neq y$ os indicadores são independentes. Então

$$\text{Var}(\, X_i \,) = \sum_{x \in S} \frac{1}{n}\Big(1 - \frac{1}{n}\Big) = c\Big(1 - \frac{1}{n}\Big).$$

Chebyshev:

$$\mathbb{P}[\, |X_i - c| \geq t \,] \leq \frac{c(1 - 1/n)}{t^2} \leq \frac{c}{t^2}.$$

Variância do tamanho de uma lista

Seja $X_i = \sum_{x \in S} \mathbb{1}_{[h(x)=i]}$, com $\mathbb{E}X_i = c$. Para $x \neq y$ os indicadores são independentes. Então

$$\mathrm{Var}(\, X_i \,) = \sum_{x \in S} \frac{1}{n}\Big(1 - \frac{1}{n}\Big) = c\Big(1 - \frac{1}{n}\Big).$$

Chebyshev:

$$\mathbb{P}[\, |X_i - c| \geq t \,] \leq \frac{c(1 - 1/n)}{t^2} \leq \frac{c}{t^2}.$$

Com $t = c$: $\mathbb{P}[X_i \geq 2c] \leq \frac{1}{c}$, só útil para c grande.

Observação-chave

Esta conta só usa que os indicadores $\mathbb{1}_{[h(x)=i]}$ são independentes dois a dois.

Maior lista via Chebyshev

Cota da união sobre as n listas:

$$\mathbb{P}\big[\max_i X_i \geq c + t\big] \leq n \cdot \frac{c}{t^2} = \frac{m}{t^2}.$$

Com $t = \sqrt{2m}$, a maior lista tem tamanho

$$\leq c + \sqrt{2m} = O(\sqrt{m})$$

com probabilidade pelo menos $1/2$.

Maior lista via Chebyshev

Cota da união sobre as n listas:

$$\mathbb{P}\big[\max_i X_i \geq c + t\big] \leq n \cdot \frac{c}{t^2} = \frac{m}{t^2}.$$

Com $t = \sqrt{2m}$, a maior lista tem tamanho

$$\leq c + \sqrt{2m} = O(\sqrt{m})$$

com probabilidade pelo menos $1/2$.

Comparação

Método	**Cota**	**Independência usada**
Cota da união sobre T	$O(\log n / \log\log n)$	muitas variáveis
Chebyshev	$O(\sqrt{m})$	só pares

Mais fraca, mas não exige "muita independência".

Variância do número de colisões

$C = \sum_{\{x,y\}} \mathbb{1}_{[h(x)=h(y)]}$. Para pares distintos $\{x, y\} \neq \{z, w\}$:

- disjuntos: dependem de quatro valores independentes;
- com elemento comum ($\{x, y\}$ e $\{x, z\}$):

$$\mathbb{P}[h(x) = h(y) = h(z)] = \sum_{a \in N} \frac{1}{n^3} = \frac{1}{n^2} = \mathbb{P}[h(x) = h(y)]\,\mathbb{P}[h(x) = h(z)].$$

Variância do número de colisões

$C = \sum_{\{x,y\}} \mathbb{1}_{[h(x)=h(y)]}$. Para pares distintos $\{x, y\} \neq \{z, w\}$:

- disjuntos: dependem de quatro valores independentes;
- com elemento comum ($\{x, y\}$ e $\{x, z\}$):

$$\mathbb{P}[h(x) = h(y) = h(z)] = \sum_{a \in N} \frac{1}{n^3} = \frac{1}{n^2} = \mathbb{P}[h(x) = h(y)]\,\mathbb{P}[h(x) = h(z)].$$

Os indicadores são independentes dois a dois, logo $\mathrm{Var}(C) = \binom{m}{2}\left(\frac{1}{n} - \frac{1}{n^2}\right)$ e, por Chebyshev,

$$\mathbb{P}[C = 0] \leq \frac{\mathrm{Var}(C)}{(\mathbb{E}C)^2} = \frac{n(1 - 1/n)}{\binom{m}{2}} \approx \frac{2n}{m^2}.$$

Para $m \gg \sqrt{n}$ há colisão com alta probabilidade, complementando o limiar do paradoxo dos aniversários.

Cuidado: onde a independência de ordem maior entra

Na conta acima, a igualdade

$$\mathbb{P}[h(x) = h(y) = h(z)] = \mathbb{P}[h(x) = h(y)]\,\mathbb{P}[h(x) = h(z)]$$

usa a independência de três valores $h(x), h(y), h(z)$, e o caso de pares disjuntos usa quatro.

Cuidado: onde a independência de ordem maior entra

Na conta acima, a igualdade

$$\mathbb{P}[h(x) = h(y) = h(z)] = \mathbb{P}[h(x) = h(y)]\,\mathbb{P}[h(x) = h(z)]$$

usa a independência de três valores $h(x), h(y), h(z)$, e o caso de pares disjuntos usa quatro.

Consequência

A fórmula para Var(C) vale para funções sorteadas em famílias com independência de ordem ≥ 4.

Já a variância de X_i usa apenas independência par a par.

O que usamos de cada propriedade

Resultado	**Propriedade necessária**
$\mathbb{E}\, C = \binom{m}{2}/n$, tempo esperado de busca	$\mathbb{P}[h(x) = h(y)] \leq 1/n$ (pares)
$\mathrm{Var}(\, X_i\,)$, maior lista $O(\sqrt{m})$	independência par a par
$\mathrm{Var}(\, C\,) = \binom{m}{2}(1/n - 1/n^2)$	independência de ordem 4
maior lista $\Theta(\log n / \log\log n)$	independência de muitas variáveis

O que usamos de cada propriedade

Resultado	**Propriedade necessária**
$\mathbb{E}\, C = \binom{m}{2}/n$, tempo esperado de busca	$\mathbb{P}[h(x) = h(y)] \leq 1/n$ (pares)
$\mathrm{Var}(\, X_i\,)$, maior lista $O(\sqrt{m})$	independência par a par
$\mathrm{Var}(\, C\,) = \binom{m}{2}(1/n - 1/n^2)$	independência de ordem 4
maior lista $\Theta(\log n / \log\log n)$	independência de muitas variáveis

Ideia central

Qualquer família em que dois elementos distintos colidem com probabilidade $\leq 1/n$ tem, no máximo, o mesmo número esperado de colisões que uma função aleatória.

Para onde vamos: famílias universais

Funções aleatórias não são práticas: descrição de tamanho $\approx |U|$.

Para onde vamos: famílias universais

Funções aleatórias não são práticas: descrição de tamanho $\approx |U|$.

A meta

Uma família $\mathcal{H} \subseteq N^U$ *pequena* tal que, para $h \in \mathcal{H}$ uniforme,

$$\mathbb{P}_h[h(x) = h(y)] \leq \frac{1}{n} \qquad \text{para todos } x \neq y \text{ em } U,$$

com funções fáceis de computar e descrição curta.

Para onde vamos: famílias universais

Funções aleatórias não são práticas: descrição de tamanho $\approx |U|$.

A meta

Uma família $\mathcal{H} \subseteq N^U$ *pequena* tal que, para $h \in \mathcal{H}$ uniforme,

$$\mathbb{P}_h[h(x) = h(y)] \leq \frac{1}{n} \qquad \text{para todos } x \neq y \text{ em } U,$$

com funções fáceis de computar e descrição curta.

- Comportar-se como função aleatória é propriedade da *família*, não de uma função isolada.
- Tudo que depende só de pares continuará valendo (esperança de colisões, tempo esperado de busca, $\mathrm{Var}(X_i)$).

A ideia: reduzir a aleatoriedade

Desejamos funções que

- tenham representação sucinta;
- imitem uma função aleatória nas estatísticas de interesse (busca em tempo constante, ausência de listas longas);
- sejam calculadas eficientemente em cada ponto.

Estratégia

Em vez de sortear h em N^U, sortear h de uma coleção menor e não vazia $\mathcal{H} \subseteq N^U$.

A condição $\mathbb{P}[h(x) = i] = \frac{1}{n}$ (para todo x e todo i) é substituída por uma condição só sobre pares.

hash universal

Família universal

Definição 4

Uma família não vazia $\mathcal{H} \subseteq N^U$ é universal se

$$\mathbb{P}_{h \in \mathcal{H}}[h(x) = h(y)] \;\leq\; \frac{1}{n} \qquad \text{para todos } x \neq y \text{ em } U.$$

- Para a função totalmente aleatória vale a igualdade, $= 1/n$.
- Carter e Wegman (1979): em vez de exigir independência total entre as imagens de todas as chaves, o que consome muitos bits aleatórios, basta controlar a colisão entre *quaisquer duas* chaves.
- Famílias com independência de pares também são universais; a recíproca é falsa.

O que continua valendo (como desigualdades)

As contas dos slides anteriores que usam apenas $\mathbb{P}[h(x) = h(y)] \leq 1/n$ valem para $\mathcal{H}$ universal:

- tamanho esperado da lista $H[h(x)]$;
- tempo esperado de busca;
- número esperado de colisões.

Porém

O que *não* se transfere é o que usava independência de ordem maior: Var(C) e a cota $\Theta(\log n / \log \log n)$ da maior lista.

Teorema: colisões com um elemento fixo

Teorema 5

Se $\mathcal{H}$ é universal, $S \subseteq U$ com $|S| = m$ e $x \in U$, então para h escolhida em $\mathcal{H}$, o número esperado de colisões entre x e os elementos de $S \setminus \{x\}$ é no máximo m/n.

Teorema: colisões com um elemento fixo

Teorema 5

Se $\mathcal{H}$ é universal, $S \subseteq U$ com $|S| = m$ e $x \in U$, então para h escolhida em $\mathcal{H}$, o número esperado de colisões entre x e os elementos de $S \setminus \{x\}$ é no máximo m/n.

Proof.

Linearidade da esperança e universalidade:

$$\mathbb{E} \sum_{y \in S \setminus \{x\}} \mathbb{1}_{[h(x)=h(y)]} = \sum_{y \in S \setminus \{x\}} \mathbb{P}[h(x) = h(y)] \leq \frac{|S \setminus \{x\}|}{n} \leq \frac{m}{n}. \qquad \square$$

Universal não implica uniforme

$U = \{1, 2\}$, $N = \{0, 1\}$

	1	2
f_1	0	0
f_2	1	0
f_3	0	1

- $\mathbb{P}[f(1) = f(2)] = \frac{1}{3} \leq \frac{1}{2}$: a família é universal.

Universal não implica uniforme

$U = \{1, 2\}$, $N = \{0, 1\}$

	1	2
f_1	0	0
f_2	1	0
f_3	0	1

- $\mathbb{P}[f(1) = f(2)] = \frac{1}{3} \leq \frac{1}{2}$: a família é universal.
- Mas $\mathbb{P}[f(1) = 0] = \frac{2}{3} \neq \frac{1}{2}$: cada $h(x)$ não é uniforme em N.

Universal não implica uniforme

$U = \{1, 2\}$, $N = \{0, 1\}$

	1	2
f_1	0	0
f_2	1	0
f_3	0	1

- $\mathbb{P}[f(1) = f(2)] = \frac{1}{3} \leq \frac{1}{2}$: a família é universal.
- Mas $\mathbb{P}[f(1) = 0] = \frac{2}{3} \neq \frac{1}{2}$: cada $h(x)$ não é uniforme em N.

Lição

Universalidade é uma propriedade sobre *pares*, não sobre cada valor $h(x)$ isoladamente.

Uma família universal linear

$U = \{0,1\}^d$, $N = \{0,1\}^k$ ($|U| = 2^d$, $|N| = 2^k$).

Definição

Sorteie os bits de A ($k \times d$) e defina $h(\mathrm{x}) = \mathrm{A} \cdot \mathrm{x}$, com soma e produto módulo 2.

- **Descrição:** só a matriz, $kd = O(\log|N| \cdot \log|U|)$ bits.
- **Não é uniforme:** $h(0) = 0$ para toda escolha de A.
- **É universal**.

A família linear é universal

Sejam $x \neq y$. Por linearidade, $h(x) = h(y) \iff A(x - y) = 0$, isto é,

$$\sum_{j=1}^{d} a_{\ell,j}(x - y)_j = 0 \quad \text{para cada } \ell = 1, \ldots, k.$$

Seja r com $(x - y)_r = 1$. Fixe todos os bits de A fora da coluna r. **Para cada** ℓ, a equação vale se, e somente se,

$$a_{\ell,r} = \sum_{j \neq r} a_{\ell,j}(x - y)_j \pmod 2,$$

que fixa um único valor para $a_{\ell,r}$.
Os k bits da coluna r são independentes e uniformes, logo

$$\mathbb{P}[Ax = Ay] = 2^{-k} = \frac{1}{|N|}. \qquad \blacksquare$$

O exemplo canônico: $h_{(a,b)}$

$U = \{0, \ldots, u-1\}$, $N = \{0, \ldots, n-1\}$, $u > n$, p primo com $u < p \leq 2u$

Família $\mathcal{H}_{p,n}$

$$h_{(a,b)}(x) = \big((ax + b) \bmod p\big) \bmod n, \qquad a \in \{1, \ldots, p-1\},\ b \in \{0, \ldots, p-1\}.$$

Sortear h é sortear (a, b) uniformemente: $p(p-1)$ escolhas.

Com $a = 0$ a função é constante e todos os pares colidem.

O exemplo canônico: $h_{(a,b)}$

$U = \{0, \ldots, u-1\}$, $N = \{0, \ldots, n-1\}$, $u > n$, p primo com $u < p \leq 2u$

Família $\mathcal{H}_{p,n}$

$$h_{(a,b)}(x) = \big((ax + b) \bmod p\big) \bmod n, \qquad a \in \{1, \ldots, p-1\},\ b \in \{0, \ldots, p-1\}.$$

Sortear h é sortear (a, b) uniformemente: $p(p-1)$ escolhas.

Com $a = 0$ a função é constante e todos os pares colidem.

Plano da prova ($x \neq y$ fixos):

1. módulo p nunca há colisão, e $(a, b) \leftrightarrow (i, j)$ é bijeção;
2. contar os pares (i, j) distintos com $i \equiv j \pmod{n}$.

Passo 1: nenhuma colisão módulo p; bijeção

Como $u < p$, $0 < |x - y| < p$ e, sendo p primo, $p \nmid x - y$. Para $a \neq 0$, $p \nmid a(x - y)$, logo

$$(ax + b) \bmod p \;\neq\; (ay + b) \bmod p.$$

Passo 1: nenhuma colisão módulo p; bijeção

Como $u < p$, $0 < |x - y| < p$ e, sendo p primo, $p \nmid x - y$. Para $a \neq 0$, $p \nmid a(x - y)$, logo

$$(ax + b) \bmod p \;\neq\; (ay + b) \bmod p.$$

Para $i, j \in \{0, \ldots, p-1\}$ distintos e $x, y \in U$ distintos, o sistema

$$\begin{pmatrix} x & 1 \\ y & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} \equiv \begin{pmatrix} i \\ j \end{pmatrix} \pmod{p}$$

tem exatamente uma solução (a, b) (pois $\det \begin{pmatrix} x & 1 \\ y & 1 \end{pmatrix} = x - y \neq 0$) com $a \neq 0$ porque $a(x - y) \equiv i - j \not\equiv 0$.

Passo 1: nenhuma colisão módulo p; bijeção

Como $u < p$, $0 < |x - y| < p$ e, sendo p primo, $p \nmid x - y$. Para $a \neq 0$, $p \nmid a(x - y)$, logo

$$(ax + b) \bmod p \;\neq\; (ay + b) \bmod p.$$

Para $i, j \in \{0, \ldots, p-1\}$ distintos e $x, y \in U$ distintos, o sistema

$$\begin{pmatrix} x & 1 \\ y & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} \equiv \begin{pmatrix} i \\ j \end{pmatrix} \pmod{p}$$

tem exatamente uma solução (a, b) (pois $\det \begin{pmatrix} x & 1 \\ y & 1 \end{pmatrix} = x - y \neq 0$) com $a \neq 0$ porque $a(x - y) \equiv i - j \not\equiv 0$.

Consequência

Há $p(p-1)$ pares (a, b) com $a \neq 0$ e $p(p-1)$ pares (i, j) distintos: uma bijeção.
Para (a, b) uniforme, (i, j) é uniforme entre os pares de valores distintos módulo p.

Passo 2: contar pares congruentes módulo n

Queremos o número de pares (i, j), $i \neq j$ em $\{0, \ldots, p-1\}$, com $j \equiv i \pmod{n}$.

- Fixado i, os $j \neq i$ com $j \equiv i$ correspondem aos múltiplos não nulos de n em $\{-i, \ldots, -i+p-1\}$ (a diferença $j-i$); o múltiplo 0 corresponde a $j = i$.

- Entre p inteiros consecutivos há no máximo $\lceil p/n \rceil \leq \frac{p-1}{n} + 1$ múltiplos de n. Tirando o 0, sobram no máximo $\dfrac{p-1}{n}$ valores de j.

- Portanto, há no máximo $\dfrac{p(p-1)}{n}$ pares (i, j) distintos e congruentes módulo n.

Conclusão: $\mathcal{H}_{p,n}$ é universal

Para $x \neq y$ em U:

$$\mathbb{P}[h_{(a,b)}(x) = h_{(a,b)}(y)] \;\leq\; \frac{p(p-1)/n}{p(p-1)} = \frac{1}{n}.$$

Custo da família

- **Descrição:** o par (a, b), com $2\lceil \log p \rceil \leq 2 \log u + O(1) = O(\log |U|)$ bits (p é fixo e conhecido).
- **Tempo:** no modelo RAM com palavras de $\Theta(\log u)$ bits, uma multiplicação, uma soma e dois restos: tempo constante.

Conclusão: $\mathcal{H}_{p,n}$ é universal

Para $x \neq y$ em U:

$$\mathbb{P}[h_{(a,b)}(x) = h_{(a,b)}(y)] \;\leq\; \frac{p(p-1)/n}{p(p-1)} = \frac{1}{n}.$$

Custo da família

- **Descrição:** o par (a, b), com $2\lceil \log p \rceil \leq 2 \log u + O(1) = O(\log |U|)$ bits (p é fixo e conhecido).
- **Tempo:** no modelo RAM com palavras de $\Theta(\log u)$ bits, uma multiplicação, uma soma e dois restos: tempo constante.

Compare com a função totalmente aleatória: $\approx |U| \log n$ bits.

aplicações de *hash* universal

Aplicação em *streaming*: contar frequências

Itens $x_1, x_2, \ldots, x_m \in U$ chegam *um a um* (acessos a páginas, pacotes numa rede). A qualquer momento queremos estimar a frequência

$$f(x) \coloneqq |\{\, t \colon x_t = x \,\}|, \qquad (x \in U).$$

O fluxo tem comprimento m, de modo que $\sum_y f(y) = m$.

Aplicação em *streaming*: contar frequências

Itens $x_1, x_2, \ldots, x_m \in U$ chegam *um a um* (acessos a páginas, pacotes numa rede). A qualquer momento queremos estimar a frequência

$$f(x) \coloneqq |\{\, t \colon x_t = x \,\}|, \qquad (x \in U).$$

O fluxo tem comprimento m, de modo que $\sum_y f(y) = m$.

- Um contador para cada $x \in U$: espaço $|U|$, proibitivo.
- Guardar os m itens: também pode ser proibitivo.

Aplicação em *streaming*: contar frequências

Itens $x_1, x_2, \ldots, x_m \in U$ chegam *um a um* (acessos a páginas, pacotes numa rede). A qualquer momento queremos estimar a frequência

$$f(x) \coloneqq |\{\, t : x_t = x \,\}|, \qquad (x \in U).$$

O fluxo tem comprimento m, de modo que $\sum_y f(y) = m$.

- Um contador para cada $x \in U$: espaço $|U|$, proibitivo.
- Guardar os m itens: também pode ser proibitivo.

Ideia

Resumir o fluxo numa *pequena* matriz de contadores, indexada por funções de *hash* universais. Aceitamos um erro pequeno, com probabilidade de falha pequena.

O *sketch*: d linhas, n colunas

Sejam $n, d \geq 1$ e $h_1, \ldots, h_d \colon U \to \{0, 1, \ldots, n-1\}$ sorteadas independentemente e uniformemente numa família universal $\mathcal{H}$. Matriz $C[j][k]$, inicialmente nula.

Chegada de x incrementar $C[j][h_j(x)]$, $\forall j$.

Consulta de x devolver

$$\widehat{f}(x) := \min_{1 \leq j \leq d} C[j][h_j(x)].$$

	0	1	2	3	4
j=1	5	3	7	0	4
j=2	3	5	1	6	4
j=3	9	3	0	2	5

x cai nas células destacadas:
$\widehat{f}(x) = \min\{7, 4, 9\} = 4$

Garantia do Count-Min *sketch*

Proposição 2

Para todo $x \in U$ *vale* $\widehat{f}(x) \geq f(x)$. *Além disso, dados* $\varepsilon, \delta \in (0,1)$, *se* $n = \lceil 2/\varepsilon \rceil$ *e* $d = \lceil \log_2(1/\delta) \rceil$, *então*

$$\mathbb{P}\big[\widehat{f}(x) > f(x) + \varepsilon m\big] \leq \delta.$$

O sketch usa $nd = O\big(\frac{1}{\varepsilon} \log \frac{1}{\delta}\big)$ *contadores de* $O(\log m)$ *bits, mais* $O(\log |U|)$ *bits para descrever cada uma das* d *funções.*

Garantia do Count-Min *sketch*

Proposição 2

Para todo $x \in U$ vale $\widehat{f}(x) \geq f(x)$. Além disso, dados $\varepsilon, \delta \in (0,1)$, se $n = \lceil 2/\varepsilon \rceil$ e $d = \lceil \log_2(1/\delta) \rceil$, então

$$\mathbb{P}\big[\widehat{f}(x) > f(x) + \varepsilon m\big] \leq \delta.$$

O sketch usa $nd = O\big(\frac{1}{\varepsilon} \log \frac{1}{\delta}\big)$ contadores de $O(\log m)$ bits, mais $O(\log |U|)$ bits para descrever cada uma das d funções.

- O erro é unilateral: nunca subestimamos.
- O espaço não depende de $|U|$ nem de m (salvo pelos $\log m$ bits de cada contador).

Prova (1): erro de uma linha

Fixe $x \in U$ e $C_j \coloneqq C[j][h_j(x)]$. Cada chegada de x incrementa C_j, logo $C_j \geq f(x)$ para todo j e, portanto, $\widehat{f}(x) \geq f(x)$.

Prova (1): erro de uma linha

Fixe $x \in U$ e $C_j \coloneqq C[j][h_j(x)]$. Cada chegada de x incrementa C_j, logo $C_j \geq f(x)$ para todo j e, portanto, $\widehat{f}(x) \geq f(x)$.

Erro da linha j

$$E_j \coloneqq C_j - f(x) = \sum_{y \neq x} f(y)\, \mathbb{1}_{[h_j(y)=h_j(x)]} \geq 0,$$

a soma das frequências dos outros itens que colidem com x na linha j.

Prova (1): erro de uma linha

Fixe $x \in U$ e $C_j \coloneqq C[j][h_j(x)]$. Cada chegada de x incrementa C_j, logo $C_j \geq f(x)$ para todo j e, portanto, $\widehat{f}(x) \geq f(x)$.

Erro da linha j

$$E_j \coloneqq C_j - f(x) = \sum_{y \neq x} f(y)\, \mathbb{1}_{[h_j(y)=h_j(x)]} \geq 0,$$

a soma das frequências dos outros itens que colidem com x na linha j.

Como $\mathcal{H}$ é universal, pela linearidade da esperança,

$$\mathbb{E}E_j = \sum_{y \neq x} f(y)\, \mathbb{P}[h_j(y) = h_j(x)] \leq \frac{1}{n} \sum_{y \neq x} f(y) \leq \frac{m}{n}.$$

Prova (2): Markov em uma linha

Como $E_j \geq 0$, a desigualdade de Markov dá

$$\mathbb{P}[E_j > \varepsilon m] \leq \mathbb{P}[E_j \geq \varepsilon m] \leq \frac{\mathbb{E}E_j}{\varepsilon m} \leq \frac{1}{\varepsilon n} \leq \frac{1}{2},$$

pois $n \geq 2/\varepsilon$.

Prova (2): Markov em uma linha

Como $E_j \geq 0$, a desigualdade de Markov dá

$$\mathbb{P}[E_j > \varepsilon m] \leq \mathbb{P}[E_j \geq \varepsilon m] \leq \frac{\mathbb{E}E_j}{\varepsilon m} \leq \frac{1}{\varepsilon n} \leq \frac{1}{2},$$

pois $n \geq 2/\varepsilon$.

Note que $\widehat{f}(x) = f(x) + \min_j E_j$. Logo

$$\widehat{f}(x) > f(x) + \varepsilon m \iff E_j > \varepsilon m \text{ para todo } j.$$

Prova (3): d linhas independentes

As funções $h_1, \ldots, h_d$ são independentes e E_j depende só de h_j, portanto os eventos $[E_j > \varepsilon m]$ são independentes e

$$\mathbb{P}\big[\widehat{f}(x) > f(x) + \varepsilon m\big] = \prod_{j=1}^{d} \mathbb{P}[E_j > \varepsilon m] \leq 2^{-d} \leq \delta,$$

pois $d \geq \log_2(1/\delta)$.

Prova (3): d linhas independentes

As funções $h_1, \ldots, h_d$ são independentes e E_j depende só de h_j, portanto os eventos $[E_j > \varepsilon m]$ são independentes e

$$\mathbb{P}\big[\widehat{f}(x) > f(x) + \varepsilon m\big] = \prod_{j=1}^{d} \mathbb{P}[E_j > \varepsilon m] \leq 2^{-d} \leq \delta,$$

pois $d \geq \log_2(1/\delta)$.

Contagem de espaço

$nd = \lceil 2/\varepsilon \rceil \lceil \log_2(1/\delta) \rceil = O\big(\frac{1}{\varepsilon} \log \frac{1}{\delta}\big)$ contadores, cada um com valor $\leq m$, isto é, $O(\log m)$ bits. Cada h_j tem descrição $O(\log |U|)$ bits. □

Observações

Numericamente: $\varepsilon = \delta = 0{,}01$ dá $n = 200$, $d = 7$, apenas 1400 contadores, qualquer que seja $|U|$.

Observações

Numericamente: $\varepsilon = \delta = 0{,}01$ dá $n = 200$, $d = 7$, apenas 1400 contadores, qualquer que seja $|U|$.

- **O que foi usado?** Só $\mathbb{P}[h(x) = h(y)] \leq 1/n$ e a desigualdade de Markov. Basta universalidade; não é preciso independência de pares.

Observações

Numericamente: $\varepsilon = \delta = 0{,}01$ dá $n = 200$, $d = 7$, apenas 1400 contadores, qualquer que seja $|U|$.

- **O que foi usado?** Só $\mathbb{P}[h(x) = h(y)] \leq 1/n$ e a desigualdade de Markov. Basta universalidade; não é preciso independência de pares.
- **Erro absoluto, não relativo.** O erro é εm, e não uma fração de $f(x)$: a estimativa só é informativa para itens *frequentes*, $f(x) \gg \varepsilon m$ (por exemplo, os itens mais acessados).

Observações

Numericamente: $\varepsilon = \delta = 0{,}01$ dá $n = 200$, $d = 7$, apenas 1400 contadores, qualquer que seja $|U|$.

- **O que foi usado?** Só $\mathbb{P}[h(x) = h(y)] \leq 1/n$ e a desigualdade de Markov. Basta universalidade; não é preciso independência de pares.
- **Erro absoluto, não relativo.** O erro é εm, e não uma fração de $f(x)$: a estimativa só é informativa para itens *frequentes*, $f(x) \gg \varepsilon m$ (por exemplo, os itens mais acessados).
- **Só chegadas.** Com remoções, $\widehat{f}(x) \geq f(x)$ pode falhar e a análise é outra.

Observações

Numericamente: $\varepsilon = \delta = 0{,}01$ dá $n = 200$, $d = 7$, apenas 1400 contadores, qualquer que seja $|U|$.

- **O que foi usado?** Só $\mathbb{P}[h(x) = h(y)] \leq 1/n$ e a desigualdade de Markov. Basta universalidade; não é preciso independência de pares.
- **Erro absoluto, não relativo.** O erro é εm, e não uma fração de $f(x)$: a estimativa só é informativa para itens *frequentes*, $f(x) \gg \varepsilon m$ (por exemplo, os itens mais acessados).
- **Só chegadas.** Com remoções, $\widehat{f}(x) \geq f(x)$ pode falhar e a análise é outra.

Padrão recorrente

Probabilidade de falha constante em cada tentativa, reduzida exponencialmente por repetições independentes (2^{-d}).

Filtro de Bloom: conjuntos com erro unilateral

Queremos representar $S \subseteq U$, com $m = |S|$, e responder "$x \in S$?".

- Sem erros, qualquer estrutura precisa de pelo menos $\log_2 \binom{|U|}{m} \geq m \log_2(|U|/m)$ bits (há $\binom{|U|}{m}$ conjuntos possíveis).
- Uma tabela de espalhamento guarda os elementos e usa $\approx m \log_2 |U|$ bits.

Em muitas aplicações (endereço numa lista de páginas maliciosas, chave num arquivo em disco antes de pagar o acesso) basta errar num só sentido:

falso positivo responder "sim" para $x \notin S$: *tolerado*.

falso negativo responder "não" para $x \in S$: *proibido*.

Filtro de Bloom: conjuntos com erro unilateral

Queremos representar $S \subseteq U$, com $m = |S|$, e responder "$x \in S$?".

- Sem erros, qualquer estrutura precisa de pelo menos $\log_2 \binom{|U|}{m} \geq m \log_2(|U|/m)$ bits (há $\binom{|U|}{m}$ conjuntos possíveis).
- Uma tabela de espalhamento guarda os elementos e usa $\approx m \log_2 |U|$ bits.

Em muitas aplicações (endereço numa lista de páginas maliciosas, chave num arquivo em disco antes de pagar o acesso) basta errar num só sentido:

falso positivo responder "sim" para $x \notin S$: *tolerado*.

falso negativo responder "não" para $x \in S$: *proibido*.

Ganho

Poucos bits por elemento, sem depender do tamanho dos elementos.

Filtros de Bloom

- Seja $S \subseteq U$ com $|S| = m$ e $h\colon U \to N$ uma função de hash uniforme ($n = |N|$ a determinar).
- Mantemos um vetor de bits A de tamanho n, inicialmente zero. Para cada $x \in S$, fazemos $A[h(x)] = 1$.
- Para testar $x \in S$, respondemos "sim" se $A[h(x)] = 1$.

Garantias:

- Sem falsos negativos: se $x \in S$, sempre respondemos "sim".
- Poucos falsos positivos: se $x \notin S$, respondemos "sim" com probabilidade $\mathbb{E}\dfrac{|\{\, i \in N\colon A[i] = 1 \,\}|}{n} = 1 - \left(1 - \frac{1}{n}\right)^m \leqslant \frac{m}{n}$.

Filtros de Bloom: escolha de n

- Com $n = 2m$, para $x \notin S$ respondemos "não" com probabilidade pelo menos $\frac{1}{2}$. Na verdade é mais, pois ao espalhar m elementos em $2m$ posições há colisões e o número de 1's fica menor que m.
- Para probabilidade $\frac{1}{2}$ basta $n \approx 1{,}44\, m$, pois

$$\mathbb{P}_h[A[i] = 0] = \mathbb{P}[h(x) \neq i,\ \forall x \in S] = \left(1 - \frac{1}{n}\right)^m \approx e^{-m/n}.$$

- Para $n = m/\ln 2 \approx 1{,}44\, m$, o número esperado de zeros em A é $n/2$.

Filtros de Bloom

- Um filtro de Bloom usa $O(m)$ bits, muito menos que os $O(m \log u)$ bits necessários sem erros.
- Pode-se mostrar que h não precisa ser uniforme: uma função de hash $O(\log m)$-independente dá as mesmas garantias e pode ser armazenada de forma muito eficiente, com apenas $O(\log^2 m)$ bits.

Usando universalidade

A estrutura: k linhas de n bits.

Sejam $n, k \geq 1$ e $h_1, \ldots, h_k : U \to \{0, 1, \ldots, n-1\}$ sorteadas independentemente e uniformemente numa família universal $\mathcal{H}$.

Matriz de bits $B[j][l]$, $1 \leq j \leq k$, $0 \leq l < n$, inicialmente nula.

Inserção de x para cada j, $B[j][h_j(x)] \leftarrow 1$.

Consulta de x devolver 1 ("talvez $x \in S$") se $B[j][h_j(x)] = 1$ para todo j;
senão, 0 ("certamente $x \notin S$").

Usando universalidade

A estrutura: k linhas de n bits.

Sejam $n, k \geq 1$ e $h_1, \ldots, h_k : U \to \{0, 1, \ldots, n-1\}$ sorteadas independentemente e uniformemente numa família universal $\mathcal{H}$.

Matriz de bits $B[j][l]$, $1 \leq j \leq k$, $0 \leq l < n$, inicialmente nula.

Inserção de x para cada j, $B[j][h_j(x)] \leftarrow 1$.

Consulta de x devolver 1 ("talvez $x \in S$") se $B[j][h_j(x)] = 1$ para todo j;
senão, 0 ("certamente $x \notin S$").

Espaço

kn bits, mais $O(\log|U|)$ bits para descrever cada h_j (na família $h_{(a,b)}$).

É o *Count-Min sketch* com contadores de um bit.

Exemplo: $k = 3$, $n = 8$

Inserimos $S = \{a, b\}$, com $(h_1(a), h_1(b)) = (1, 5)$, $(h_2(a), h_2(b)) = (3, 6)$ e $(h_3(a), h_3(b)) = (0, 4)$.

	0	1	2	3	4	5	6	7
j=1	0	1	0	0	0	1	0	0
j=2	0	0	0	1	0	0	1	0
j=3	1	0	0	0	1	0	0	0

Exemplo: $k = 3$, $n = 8$

Inserimos $S = \{a, b\}$, com $(h_1(a), h_1(b)) = (1, 5)$, $(h_2(a), h_2(b)) = (3, 6)$ e $(h_3(a), h_3(b)) = (0, 4)$.

	0	1	2	3	4	5	6	7
j=1	0	1	0	0	0	1	0	0
j=2	0	0	0	1	0	0	1	0
j=3	1	0	0	0	1	0	0	0

Consulta de x com posições $(5, 2, 4)$ (em vermelho): a linha 2 tem 0, logo "certamente $x \notin S$". Se as três fossem 1, a resposta seria "talvez", podendo ser um falso positivo.

Garantia do filtro de Bloom

Proposição 3

Sejam S, com $|S| = m$, e $x \in U$ fixados antes do sorteio das funções, e S inserido num filtro com k linhas de n bits e $h_1, \ldots, h_k$ independentes numa família universal.

1. *Se $x \in S$, a consulta devolve 1 com probabilidade 1.*
2. *Se $x \notin S$, a consulta devolve 1 com probabilidade $\leq (m/n)^k$.*

Em particular, para $n = 2m$ e $k = \lceil \log_2(1/\delta) \rceil$, a probabilidade de falso positivo é $\leq \delta$, com espaço $kn = 2m\lceil \log_2(1/\delta) \rceil$ bits.

Garantia do filtro de Bloom

Proposição 3

Sejam S, com $|S| = m$, e $x \in U$ fixados antes do sorteio das funções, e S inserido num filtro com k linhas de n bits e $h_1, \ldots, h_k$ independentes numa família universal.

1. *Se $x \in S$, a consulta devolve 1 com probabilidade 1.*
2. *Se $x \notin S$, a consulta devolve 1 com probabilidade $\leq (m/n)^k$.*

Em particular, para $n = 2m$ e $k = \lceil \log_2(1/\delta) \rceil$, a probabilidade de falso positivo é $\leq \delta$, com espaço $kn = 2m\lceil \log_2(1/\delta) \rceil$ bits.

Sem falsos negativos; falsos positivos com probabilidade controlada.

Prova (1): uma linha

Se $x \in S$: a inserção de x pôs 1 em todas as posições $B[j][h_j(x)]$, e nenhum bit volta a ser 0. Logo a consulta devolve 1.

Prova (1): uma linha

Se $x \in S$: a inserção de x pôs 1 em todas as posições $B[j][h_j(x)]$, e nenhum bit volta a ser 0. Logo a consulta devolve 1.

Se $x \notin S$: erro da linha j

Só h_j escreve na linha j. Seja

$$E_j := \big[\exists\, y \in S\colon h_j(y) = h_j(x)\big] \qquad (\text{então } B[j][h_j(x)] = 1 \iff E_j).$$

Prova (1): uma linha

Se $x \in S$: a inserção de x pôs 1 em todas as posições $B[j][h_j(x)]$, e nenhum bit volta a ser 0. Logo a consulta devolve 1.

Se $x \notin S$: erro da linha j

Só h_j escreve na linha j. Seja

$$E_j := \big[\exists\, y \in S\colon h_j(y) = h_j(x)\big] \qquad (\text{então } B[j][h_j(x)] = 1 \iff E_j).$$

Como $x \neq y$ para todo $y \in S$, a cota da união e a universalidade dão

$$\mathbb{P}(E_j) \leq \sum_{y \in S} \mathbb{P}[h_j(y) = h_j(x)] \leq \frac{m}{n}.$$

Prova (2): k linhas independentes

S e x estão fixos e E_j depende só de h_j; as funções são independentes, logo $E_1, \ldots, E_k$ são independentes. A consulta devolve 1 se, e somente se, todos ocorrem:

$$\mathbb{P}[\text{falso positivo}] = \prod_{j=1}^{k} \mathbb{P}(E_j) \leq \left(\frac{m}{n}\right)^k.$$

Prova (2): k linhas independentes

S e x estão fixos e E_j depende só de h_j; as funções são independentes, logo $E_1, \ldots, E_k$ são independentes. A consulta devolve 1 se, e somente se, todos ocorrem:

$$\mathbb{P}[\text{falso positivo}] = \prod_{j=1}^{k} \mathbb{P}(E_j) \leq \left(\frac{m}{n}\right)^k.$$

Para $n = 2m$, a cota é $2^{-k} \leq \delta$ se $k \geq \log_2(1/\delta)$. □

Prova (2): k linhas independentes

S e x estão fixos e E_j depende só de h_j; as funções são independentes, logo $E_1, \ldots, E_k$ são independentes. A consulta devolve 1 se, e somente se, todos ocorrem:

$$\mathbb{P}[\text{falso positivo}] = \prod_{j=1}^{k} \mathbb{P}(E_j) \leq \left(\frac{m}{n}\right)^k.$$

Para $n = 2m$, a cota é $2^{-k} \leq \delta$ se $k \geq \log_2(1/\delta)$. □

O padrão de sempre

Probabilidade de falha $\leq 1/2$ em cada linha (cota da união + universalidade), reduzida exponencialmente por k repetições independentes.

A cota só é informativa se $n > m$.

Relação com o *Count-Min sketch*

Count-Min

- contadores de $O(\log m)$ bits
- consulta: $\min_j C[j][h_j(x)]$
- erro: superestima, até εm

Bloom

- contadores de 1 bit, saturando em 1
- consulta: $\bigwedge_j B[j][h_j(x)]$
- erro: falso positivo, $\leq (m/n)^k$

Relação com o *Count-Min sketch*

Count-Min

- contadores de $O(\log m)$ bits
- consulta: $\min_j C[j][h_j(x)]$
- erro: superestima, até εm

Bloom

- contadores de 1 bit, saturando em 1
- consulta: $\bigwedge_j B[j][h_j(x)]$
- erro: falso positivo, $\leq (m/n)^k$

Com contadores saturados em 1, o Count-Min vira o Bloom: o min dos bits $B[j][h_j(x)]$ é o seu AND. A análise tem a mesma forma: universalidade de cada h_j dá uma cota por linha (Markov no Count-Min, cota da união no Bloom), e a independência entre as k linhas eleva essa cota à k-ésima potência.

Taxa de falsos positivos de um filtro construído

A Proposição trata de x fixo; a probabilidade é sobre as funções. Na prática, o filtro é construído uma vez e consultado muitas vezes.

Taxa de falsos positivos de um filtro construído

A Proposição trata de x fixo; a probabilidade é sobre as funções. Na prática, o filtro é construído uma vez e consultado muitas vezes.

Corolário 6

Sob as hipóteses da Proposição, seja Q uma distribuição sobre $U \setminus S$ fixada independentemente de $h_1, \ldots, h_k$, e $\Phi := \mathbb{P}_{x \sim Q}[$a consulta de x devolve 1$]$, uma variável aleatória nas funções $h_1, \ldots, h_k$. Então $\mathbb{E}\Phi \leq (m/n)^k$ e, para todo $c > 0$,

$$\mathbb{P}\big[\Phi \geq c\,(m/n)^k\big] \leq \frac{1}{c}.$$

Taxa de falsos positivos de um filtro construído

A Proposição trata de x fixo; a probabilidade é sobre as funções. Na prática, o filtro é construído uma vez e consultado muitas vezes.

Corolário 6

Sob as hipóteses da Proposição, seja Q uma distribuição sobre $U \setminus S$ fixada independentemente de $h_1, \ldots, h_k$, e $\Phi := \mathbb{P}_{x \sim Q}[$a consulta de x devolve 1$]$, uma variável aleatória nas funções $h_1, \ldots, h_k$. Então $\mathbb{E}\Phi \leq (m/n)^k$ e, para todo $c > 0$,

$$\mathbb{P}\big[\Phi \geq c\,(m/n)^k\big] \leq \frac{1}{c}.$$

Prova. $\Phi = \sum_{x \notin S} Q(x)\, \mathbb{1}_{[x \text{ é falso positivo}]}$; pela linearidade e pela Proposição, $\mathbb{E}\Phi \leq (m/n)^k$. Como $\Phi \geq 0$, Markov dá o resto. □

Taxa de falsos positivos de um filtro construído

A Proposição trata de x fixo; a probabilidade é sobre as funções. Na prática, o filtro é construído uma vez e consultado muitas vezes.

Corolário 6

Sob as hipóteses da Proposição, seja Q uma distribuição sobre $U \setminus S$ fixada independentemente de $h_1, \ldots, h_k$, e $\Phi := \mathbb{P}_{x \sim Q}[$a consulta de x devolve 1$]$, uma variável aleatória nas funções $h_1, \ldots, h_k$. Então $\mathbb{E}\Phi \leq (m/n)^k$ e, para todo $c > 0$,

$$\mathbb{P}\big[\Phi \geq c\,(m/n)^k\big] \leq \frac{1}{c}.$$

Prova. $\Phi = \sum_{x \notin S} Q(x)\, \mathbb{1}_{[x \text{ é falso positivo}]}$; pela linearidade e pela Proposição, $\mathbb{E}\Phi \leq (m/n)^k$. Como $\Phi \geq 0$, Markov dá o resto. □

Exemplo

Com $n = 2m$ e $k = \lceil \log_2(1/\delta) \rceil$: com probabilidade $\geq 0{,}9$ sobre as funções, a taxa de falsos positivos é $\leq 10\,\delta$.

Dimensionando o filtro (vetor compartilhado)

Com um único vetor de N bits compartilhado pelas k funções (modelo: h_j totalmente aleatórias), o erro é $\approx (1 - e^{-km/N})^k$, minimizado em $k = (N/m) \ln 2$, onde vale 2^{-k}. Para erro $\approx \delta$: $k = \log_2(1/\delta)$ e

$$N = \frac{m \log_2(1/\delta)}{\ln 2} \approx 1{,}44\, m \log_2(1/\delta) \text{ bits},$$

contra $2m\lceil \log_2(1/\delta) \rceil$ para k linhas com cota provada.

Dimensionando o filtro (vetor compartilhado)

Com um único vetor de N bits compartilhado pelas k funções (modelo: h_j totalmente aleatórias), o erro é $\approx (1 - e^{-km/N})^k$, minimizado em $k = (N/m)\ln 2$, onde vale 2^{-k}. Para erro $\approx \delta$: $k = \log_2(1/\delta)$ e

$$N = \frac{m \log_2(1/\delta)}{\ln 2} \approx 1{,}44\, m \log_2(1/\delta) \text{ bits},$$

contra $2m\lceil \log_2(1/\delta) \rceil$ para k linhas com cota provada.

- $\delta = 1\%$: cerca de 9,6 bits por elemento ($k = 7$).
- $\delta = 0{,}1\%$: cerca de 14,4 bits por elemento ($k = 10$).

Dimensionando o filtro (vetor compartilhado)

Com um único vetor de N bits compartilhado pelas k funções (modelo: h_j totalmente aleatórias), o erro é $\approx (1 - e^{-km/N})^k$, minimizado em $k = (N/m)\ln 2$, onde vale 2^{-k}. Para erro $\approx \delta$: $k = \log_2(1/\delta)$ e

$$N = \frac{m \log_2(1/\delta)}{\ln 2} \approx 1{,}44\, m \log_2(1/\delta) \text{ bits},$$

contra $2m\lceil \log_2(1/\delta) \rceil$ para k linhas com cota provada.

- $\delta = 1\%$: cerca de 9,6 bits por elemento ($k = 7$).
- $\delta = 0{,}1\%$: cerca de 14,4 bits por elemento ($k = 10$).

O espaço por elemento depende só de δ, não de m nem de $|U|$. Qualquer estrutura com erro δ precisa de $\approx m \log_2(1/\delta)$ bits, então o Bloom fica a um fator $\log_2 e \approx 1{,}44$ do ótimo. Na prática escolhe-se N um pouco maior, pois o erro real excede a fórmula.

Exemplo: comparação de espaço

$m = 10^8$ endereços de 50 bytes (400 bits) e $\delta = 10^{-2}$.

estrutura	bits	aprox.
guardar os endereços	4×10^{10}	5 GB
impressões de 34 bits	$3{,}4 \times 10^9$	425 MB
Bloom em k linhas, $n = 2m$, $k = 7$	$1{,}4 \times 10^9$	175 MB
Bloom em vetor único, $k = 7$	$9{,}6 \times 10^8$	120 MB

Exemplo: comparação de espaço

$m = 10^8$ endereços de 50 bytes (400 bits) e $\delta = 10^{-2}$.

estrutura	bits	aprox.
guardar os endereços	4×10^{10}	5 GB
impressões de 34 bits	$3{,}4 \times 10^9$	425 MB
Bloom em k linhas, $n = 2m$, $k = 7$	$1{,}4 \times 10^9$	175 MB
Bloom em vetor único, $k = 7$	$9{,}6 \times 10^8$	120 MB

Impressões: cada endereço vira $r = \lceil \log_2(m/\delta) \rceil = 34$ bits por uma h universal com contradomínio 2^r; pela cota da união, a probabilidade de $x \notin S$ coincidir com alguma impressão é $\leq m2^{-r} \leq \delta$. O custo mr cresce com $\log m$ e não inclui a estrutura para guardar e consultar as impressões.

Observações

- **Ótimo:** qualquer estrutura sem falsos negativos e com falso positivo $\leq \delta$ precisa, para $m \ll |U|$, de cerca de $m \log_2(1/\delta)$ bits (clássico; não provamos). O vetor único está a um fator $\log_2 e \approx 1{,}44$ do ótimo.

Observações

- **Ótimo:** qualquer estrutura sem falsos negativos e com falso positivo $\leq \delta$ precisa, para $m \ll |U|$, de cerca de $m \log_2(1/\delta)$ bits (clássico; não provamos). O vetor único está a um fator $\log_2 e \approx 1{,}44$ do ótimo.
- **União:** filtros com as mesmas funções e o mesmo tamanho se unem pelo OR bit a bit.

Observações

- **Ótimo:** qualquer estrutura sem falsos negativos e com falso positivo $\leq \delta$ precisa, para $m \ll |U|$, de cerca de $m \log_2(1/\delta)$ bits (clássico; não provamos). O vetor único está a um fator $\log_2 e \approx 1{,}44$ do ótimo.
- **União:** filtros com as mesmas funções e o mesmo tamanho se unem pelo OR bit a bit.
- **Sem remoção:** zerar um bit pode apagar outros elementos e criar falsos negativos. Com contadores no lugar de bits, volta-se ao *Count-Min sketch*.

Observações

- **Ótimo:** qualquer estrutura sem falsos negativos e com falso positivo $\leq \delta$ precisa, para $m \ll |U|$, de cerca de $m \log_2(1/\delta)$ bits (clássico; não provamos). O vetor único está a um fator $\log_2 e \approx 1{,}44$ do ótimo.
- **União:** filtros com as mesmas funções e o mesmo tamanho se unem pelo OR bit a bit.
- **Sem remoção:** zerar um bit pode apagar outros elementos e criar falsos negativos. Com contadores no lugar de bits, volta-se ao *Count-Min sketch*.
- **Capacidade:** a garantia vale para até m elementos; mais que isso degrada o erro. A estrutura não lista os elementos de S.

Aplicações

Uma consulta negativa barata evita uma operação cara.

- **B.D. em arquivos ordenados** (Bigtable, HBase, Cassandra): filtro das chaves de cada arquivo em memória; "certamente não" dispensa o acesso a disco.
- ***Caches*** de rede e de conteúdo; ***crawlers*** (descartar endereços já visitados).
- **Junções distribuídas:** um nó envia um filtro das chaves; só as linhas candidatas são transmitidas.
- **Bioinformática:** conjuntos de k-mers de genomas.

Aplicações

Uma consulta negativa barata evita uma operação cara.

- **B.D. em arquivos ordenados** (Bigtable, HBase, Cassandra): filtro das chaves de cada arquivo em memória; "certamente não" dispensa o acesso a disco.
- ***Caches*** de rede e de conteúdo; ***crawlers*** (descartar endereços já visitados).
- **Junções distribuídas:** um nó envia um filtro das chaves; só as linhas candidatas são transmitidas.
- **Bioinformática:** conjuntos de k-mers de genomas.

A troca central

Aceitar uma probabilidade pequena e controlável δ de erro unilateral por cerca de $1{,}44 \log_2(1/\delta)$ bits por elemento, independentemente do tamanho dos elementos.

Hashing perfeito: conjuntos estáticos

Definição

Uma função de *hash* é perfeita para um conjunto *estático* S se não há colisões em S, isto é, h é injetora em S.

Hashing perfeito: conjuntos estáticos

Definição

Uma função de *hash* é perfeita para um conjunto *estático* S se não há colisões em S, isto é, h é injetora em S.

Então cada lista tem no máximo um elemento e uma busca custa $O(1)$ no pior caso.

Hashing perfeito: conjuntos estáticos

Definição

Uma função de *hash* é perfeita para um conjunto *estático* S se não há colisões em S, isto é, h é injetora em S.

Então cada lista tem no máximo um elemento e uma busca custa $O(1)$ no pior caso.

- S não muda (só buscas), com $m = |S|$.
- Supomos $|U| > m^2$: as famílias $h_{(a,b)}$ valem para contradomínios de até m^2 posições.

Hashing perfeito: conjuntos estáticos

Definição

Uma função de *hash* é perfeita para um conjunto *estático* S se não há colisões em S, isto é, h é injetora em S.

Então cada lista tem no máximo um elemento e uma busca custa $O(1)$ no pior caso.

- S não muda (só buscas), com $m = |S|$.
- Supomos $|U| > m^2$: as famílias $h_{(a,b)}$ valem para contradomínios de até m^2 posições.

Objetivo: tempo de busca $O(1)$ no pior caso com espaço $O(m)$. Primeiro, uma construção mais simples, com espaço $O(m^2)$.

Primeira construção: $n = m^2$

Se $\mathcal{H}$ é universal e $n = m^2$, o número C de colisões em S satisfaz

$$\mathbb{E}C \leq \binom{m}{2} \frac{1}{m^2} = \frac{m(m-1)}{2m^2} < \frac{1}{2}.$$

Primeira construção: $n = m^2$

Se $\mathcal{H}$ é universal e $n = m^2$, o número C de colisões em S satisfaz

$$\mathbb{E}C \leq \binom{m}{2} \frac{1}{m^2} = \frac{m(m-1)}{2m^2} < \frac{1}{2}.$$

Por Markov, $\mathbb{P}[C \geq 1] \leq \mathbb{E}C < \frac{1}{2}$. Logo, ao sortear $h \in \mathcal{H}$,

$$\mathbb{P}[h \text{ é perfeita para } S] = \mathbb{P}[C = 0] > \tfrac{1}{2}.$$

Primeira construção: $n = m^2$

Se $\mathcal{H}$ é universal e $n = m^2$, o número C de colisões em S satisfaz

$$\mathbb{E}C \leq \binom{m}{2} \frac{1}{m^2} = \frac{m(m-1)}{2m^2} < \frac{1}{2}.$$

Por Markov, $\mathbb{P}[C \geq 1] \leq \mathbb{E}C < \frac{1}{2}$. Logo, ao sortear $h \in \mathcal{H}$,

$$\mathbb{P}[h \text{ é perfeita para } S] = \mathbb{P}[C = 0] > \tfrac{1}{2}.$$

Probabilidade de sucesso constante

Isso sugere um algoritmo do tipo Las Vegas: sorteie, teste e repita.

Algoritmo Las Vegas

1. sorteie h em $\mathcal{H}$;
2. teste todos os $O(m^2)$ pares de elementos de S para colisão;
3. repita os passos 1 e 2 até encontrar h sem colisões.

Algoritmo Las Vegas

1. sorteie h em $\mathcal{H}$;
2. teste todos os $O(m^2)$ pares de elementos de S para colisão;
3. repita os passos 1 e 2 até encontrar h sem colisões.

- Cada tentativa custa $O(m^2) = O(n)$ e tem sucesso com probabilidade $> \frac{1}{2}$.
- O número de tentativas é geométrico, de média < 2.

Algoritmo Las Vegas

1. sorteie h em $\mathcal{H}$;
2. teste todos os $O(m^2)$ pares de elementos de S para colisão;
3. repita os passos 1 e 2 até encontrar h sem colisões.

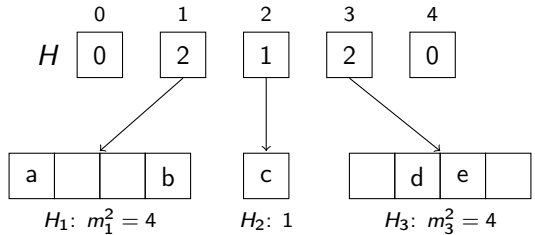
- Cada tentativa custa $O(m^2) = O(n)$ e tem sucesso com probabilidade $> \frac{1}{2}$.
- O número de tentativas é geométrico, de média < 2.

Resultado

Função perfeita em tempo *esperado* $O(n)$: busca $O(1)$ no pior caso, espaço $O(m^2)$.

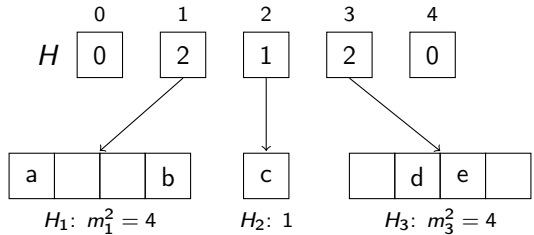
Segunda construção: dois níveis, espaço $O(m)$

- **1º nível:** espalhamos S numa tabela H de tamanho m com h universal; haverá colisões.
- **2º nível:** se m_i elementos caem na posição i, guardamo-os em H_i com m_i^2 posições, com h_i sem colisões (primeira construção). Se $m_i = 0$ não há tabela.



Segunda construção: dois níveis, espaço $O(m)$

- **1º nível:** espalhamos S numa tabela H de tamanho m com h universal; haverá colisões.
- **2º nível:** se m_i elementos caem na posição i, guardamo-os em H_i com m_i^2 posições, com h_i sem colisões (primeira construção). Se $m_i = 0$ não há tabela.



Busca de x: $i = h(x)$; se $m_i = 0$, $x \notin S$; senão compare x com $H_i[h_i(x)]$. Custo $O(1)$ no pior caso.

Espaço: $\sum_i m_i^2$ (1)

Como $m_i^2 = m_i + m_i(m_i - 1) = m_i + 2\binom{m_i}{2}$ e $\sum_i m_i = m$,

$$\mathbb{E} \sum_{i=0}^{m-1} m_i^2 = m + 2\,\mathbb{E} \sum_{i=0}^{m-1} \binom{m_i}{2}.$$

Espaço: $\sum_i m_i^2$ (1)

Como $m_i^2 = m_i + m_i(m_i - 1) = m_i + 2\binom{m_i}{2}$ e $\sum_i m_i = m$,

$$\mathbb{E} \sum_{i=0}^{m-1} m_i^2 = m + 2\,\mathbb{E} \sum_{i=0}^{m-1} \binom{m_i}{2}.$$

$\binom{m_i}{2}$ conta os pares que colidem na posição i; logo $\sum_i \binom{m_i}{2}$ é o número de colisões do 1º nível, e cada par que colide é contado uma vez. Como $\mathbb{P}[\text{par colidir}] \leq 1/m$,

$$\mathbb{E} \sum_i \binom{m_i}{2} \leq \frac{1}{m}\binom{m}{2} = \frac{m-1}{2}.$$

Espaço: $\sum_i m_i^2$ (1)

Como $m_i^2 = m_i + m_i(m_i - 1) = m_i + 2\binom{m_i}{2}$ e $\sum_i m_i = m$,

$$\mathbb{E} \sum_{i=0}^{m-1} m_i^2 = m + 2\,\mathbb{E} \sum_{i=0}^{m-1} \binom{m_i}{2}.$$

$\binom{m_i}{2}$ conta os pares que colidem na posição i; logo $\sum_i \binom{m_i}{2}$ é o número de colisões do 1º nível, e cada par que colide é contado uma vez. Como $\mathbb{P}[\text{par colidir}] \leq 1/m$,

$$\mathbb{E} \sum_i \binom{m_i}{2} \leq \frac{1}{m}\binom{m}{2} = \frac{m-1}{2}.$$

Portanto $\mathbb{E} \sum_i m_i^2 \leq m + (m-1) < 2m$.

Espaço: $\sum_i m_i^2$ (2)

Por Markov,

$$\mathbb{P}\Big[\sum_i m_i^2 \geq 4m\Big] \leq \frac{\mathbb{E}\sum_i m_i^2}{4m} < \frac{2m}{4m} = \frac{1}{2}.$$

Espaço: $\sum_i m_i^2$ (2)

Por Markov,

$$\mathbb{P}\Big[\sum_i m_i^2 \geq 4m\Big] \leq \frac{\mathbb{E}\sum_i m_i^2}{4m} < \frac{2m}{4m} = \frac{1}{2}.$$

O padrão de novo

Com probabilidade $> \frac{1}{2}$, a soma dos tamanhos das tabelas secundárias é $< 4m$. Basta repetir o sorteio de h, como na primeira construção.

Construção e custos

1. **Achar h com $\sum_i m_i^2 < 4m$.** Cada teste calcula os m_i percorrendo S ($O(m)$); sucesso $> \frac{1}{2}$ por tentativa, logo tempo esperado $O(m)$.

Construção e custos

1. **Achar h com $\sum_i m_i^2 < 4m$.** Cada teste calcula os m_i percorrendo S ($O(m)$); sucesso $> \frac{1}{2}$ por tentativa, logo tempo esperado $O(m)$.
2. **Achar cada h_i.** Fixado h, sorteamos h_i até não haver colisões entre os m_i elementos: tempo esperado $O(m_i^2)$ (primeira construção). Total esperado $O\left(\sum_i m_i^2\right) = O(m)$, pois $\sum_i m_i^2 < 4m$ *por construção*.

Construção e custos

1. **Achar h com $\sum_i m_i^2 < 4m$.** Cada teste calcula os m_i percorrendo S ($O(m)$); sucesso $> \frac{1}{2}$ por tentativa, logo tempo esperado $O(m)$.
2. **Achar cada h_i.** Fixado h, sorteamos h_i até não haver colisões entre os m_i elementos: tempo esperado $O(m_i^2)$ (primeira construção). Total esperado $O\left(\sum_i m_i^2\right) = O(m)$, pois $\sum_i m_i^2 < 4m$ *por construção*.

Espaço

m (1º nível) $+ \sum_i m_i^2 < 4m$ (2º nível) $+ O(1)$ palavras de parâmetros (a, b), (a_i, b_i) por tabela: $O(m)$.

Resultado

Hashing perfeito (Fredman, Komlós e Szemerédi, 1984)

Para um conjunto estático S de tamanho m, construímos em tempo *esperado* $O(m)$ uma estrutura de espaço $O(m)$ com busca em tempo $O(1)$ no pior caso.

Resultado

Hashing perfeito (Fredman, Komlós e Szemerédi, 1984)

Para um conjunto estático S de tamanho m, construímos em tempo *esperado* $O(m)$ uma estrutura de espaço $O(m)$ com busca em tempo $O(1)$ no pior caso.

- Só usamos universalidade (cota de colisões por par), linearidade da esperança e Markov.
- Duas aplicações do mesmo padrão: probabilidade de sucesso $> \frac{1}{2}$ em cada tentativa, tempo esperado pela geométrica de média < 2.